



← All roadmaps

MLOPS MENA COMMUNITY

MLOps Roadmap for Seniors

Depth, system thinking, and production scale

Mid → Senior

🕒 Ongoing

📖 Pick 2–3 specializations

📖 85 free resources

👤 Junior and mid-level MLOps engineers levelling up to Senior.

[Read the original on LinkedIn ↗](#)



SPECIALIZATION 1
LLMOps



SPECIALIZATION 2
Model Optimization



SPECIALIZATION 3
Production Kubernetes
& Distributed Systems



SPECIALIZATION 4
Advanced Monitoring
& Observability

I published the Basic MLOps Roadmap last week and honestly, the response overwhelmed me. So many messages from people asking: "What's next? How do I level up from Junior/Mid to Senior?"

Here's the honest answer: becoming a Senior MLOps Engineer isn't about knowing more tools. It's about depth in specific areas, system thinking, and the ability to handle production challenges that come with real-world scale.

Like the Basic roadmap, I'm focusing heavily on open-source resources and free content. Paid options are mentioned only when there's genuinely no free alternative.



Specialization 1 — LLMOps

This is the hottest sub-specialty in 2026, and it's where I've been focusing most of my time. If you want to be in demand, master this.

Core skills you'll need:

- LLM serving at scale (vLLM, SGLang, TGI)
- RAG architecture and evaluation
- LLM observability and tracing
- Fine-tuning (LoRA, QLoRA, full fine-tuning)
- Prompt management and versioning at scale
- Agentic systems and orchestration

Hugging Face courses

All free, most with certificates.

- [NLP Course](#) — foundational, still the best starting point
- [LLM Course](#) — modern and focused
- [AI Agents Course](#) — free certificate, builds real agents with smolagents/LangGraph
- [Smol Course](#) — small models, resource-constrained fine-tuning

YouTube channels worth your time

- [Andrej Karpathy](#) — if there's one person you need to follow in this field, it's Karpathy. His "Neural Networks: Zero to Hero" series takes you from neural networks to building GPT from scratch. Mandatory viewing. Even his 1-hour "Intro to LLMs" video is worth more than most week-long courses.



Umar Jamil — genuinely underrated, with insanely deep dives into LLM internals:

FlashAttention, vLLM, LoRA, Mixture of Experts, all explained in the best resources for understanding how modern LLM infrastructure actually works under the hood.

العربية

Join the Community ↗



DeepLearning.AI short courses

All free during the platform beta.

- [Fast & Efficient LLM Inference with vLLM](#) — June 2026, brand new; hands-on with LLM Compressor + vLLM + GuideLLM benchmarking
- [Efficiently Serving LLMs](#)
- [Retrieval Augmented Generation \(RAG\) by Zain Hasan](#)
- [Building Agentic RAG with LlamaIndex](#)
- [AI Agents in LangGraph](#)
- [Building Code Agents with Hugging Face smolagents](#)

Serving and observability

vLLM is the default LLM serving solution in 2026. The docs and examples are excellent, and the codebase itself is a masterclass in LLM serving engineering. For observability, I personally use Langfuse in production at Unifonic and strongly recommend it over LangSmith for self-hosted environments.

- [vLLM documentation](#)
- [vLLM on GitHub](#)
- [Langfuse](#) — open-source LLM observability
- [Langfuse on GitHub](#)

Agent frameworks worth knowing

Pick 2-3, not all.

- [LangGraph](#) — production-ready, most flexible
- [smolagents \(Hugging Face\)](#) — lightweight, code-first



CrewAI — multi-agent, role-based

MLOps MENA
COMMUNITY

العربية



[Join the Community ↗](#)



AutoGen (Microsoft) — conversational multi-agent

- [OpenAI Agents SDK](#) — the newest option

Specialization 2 — Model Optimization

The model that worked on an A100 in training might be too slow or too expensive in production. This specialization is what makes MLOps engineers genuinely valuable, and honestly, it's underrated compared to the LLMOps hype.

Core techniques to master

Quantization — FP32 → FP16/BF16/INT8/INT4.

- Tools: bitsandbytes, GPTQ, AWQ, GGUF (for CPU inference), LLM Compressor
- Modern techniques for LLMs, each with different trade-offs

Pruning — removing weights that don't contribute to performance.

- Modern techniques: SparseGPT, Wanda for LLMs
- Older but foundational: the Lottery Ticket Hypothesis

Knowledge distillation — training small "student" models from large "teacher" models.

- Example: DistilBERT (40% smaller, 97% of BERT's performance)
- Newer: MiniLM, TinyBERT

Hardware-specific acceleration

- [TensorRT](#) — NVIDIA GPU deployment
- [TensorRT-LLM](#) — LLM-specific GPU inference



Free resources

- [MIT 6.5940: TinyML and Efficient Deep Learning Computing](#) — by Prof. Song Han. THE academic course on model optimization, with free lectures and slides. This is the course that separates people who use quantization from people who understand it.
- ["Efficient Deep Learning" by Gaurav Menghani et al.](#) — free, the most comprehensive book on this topic, written by engineers at Google.
- [Quantization Fundamentals with Younes Belkada](#) — free on DeepLearning.AI
- [Quantization in Depth](#) — free on DeepLearning.AI
- Umar Jamil's ["Quantization Explained"](#) — 3-hour deep dive
- [NVIDIA TensorRT Model Optimizer](#) — all-in-one toolkit for quantization, pruning, sparsity, and distillation

Critical papers to actually read (not just skim)

- [PagedAttention \(vLLM\) by Kwon et al.](#)
 - [FlashAttention by Tri Dao](#)
 - [FlashAttention-2](#)
 - [GPTQ](#)
 - [AWQ](#)
 - [SparseGPT](#)
 - [Wanda](#)
 - [Continuous batching \(Orca\) by Yu et al.](#)
 - [Speculative Decoding by Leviathan et al.](#)
 - [The Lottery Ticket Hypothesis](#)
-



Specialization 3 — Production Kubernetes & Distributed Systems

Senior MLOps engineers own the infrastructure, not just the pipelines. If you can't operate K8s at scale, you're not senior yet.

Skills you need to develop:

- Kubernetes fundamentals (Deployments, Services, Ingress, HPA, PDBs)
- Storage classes and persistent volumes
- ML-specific orchestration (KServe, Seldon Core, Kubeflow)
- Multi-cluster management
- GPU scheduling and the NVIDIA operator
- Resource management and pod prioritization
- Service mesh basics (Istio, Linkerd)
- Node autoscaling with Karpenter or Cluster Autoscaler

Free resources

- [TechWorld with Nana's Kubernetes crash course](#) — 4 hours, the best free introduction available
- [KodeKloud](#) — free Kubernetes course with hands-on labs
- [kubernetes.io tutorials](#) — interactive in-browser labs
- [The Linux Foundation's "Introduction to Kubernetes" \(LFS158\)](#) — free on edX
- [KServe documentation](#) — this is what you use for model serving, not raw Deployment YAMLs
- [Kubeflow documentation](#)
- [Seldon Core](#)

Certifications worth considering (paid but valuable)



Skip most other certs. These two actually mean something.

Specialization 4 — Advanced Monitoring & Observability

Beyond basic Prometheus/Grafana, seniors need to handle real production monitoring challenges. This is where most ML systems break in ways nobody predicted.

Advanced concepts to master

Data drift detection — statistical tests you should actually understand:

- Kolmogorov-Smirnov test
- Population Stability Index (PSI)
- Jensen-Shannon divergence
- Wasserstein distance
- Chi-square for categorical features

Concept drift — when the world changes around your model. Different from data drift. Common in fraud detection, recommendation systems, and any adversarial environment.

Embedding drift — critical for RAG/LLM systems. Your query embeddings can drift away from your vector database's distribution over time. If you're not monitoring this, your RAG will silently degrade.

Distributed tracing — understanding latency across microservices. OpenTelemetry is the standard.



Model performance monitoring — not just system health, but ML-specific metrics that predict when your model needs retraining.

Tools to know deeply

- [Evidently AI](#) — open-source, advanced workflows
- [Arize](#)
- [WhyLabs](#)
- [Fiddler](#)
- [Langfuse](#) — LLM-specific tracing, my personal go-to
- [OpenTelemetry](#) — distributed tracing
- [Grafana Loki](#) — log aggregation
- [Prometheus](#)

Specialization 5 — Performance & Load Testing

A senior engineer doesn't deploy anything without rigorous load testing. This is not optional.

Metrics that matter:

- **Latency (p50, p95, p99)** — the p99 is what your users actually feel
- **Throughput (RPS)** — requests per second at various load levels
- **TTFT (Time To First Token)** — critical for LLMs; this is what makes ChatGPT feel "instant"
- **TPOT (Time Per Output Token)** — generation speed after the first token



Tools

- [Locust](#) — Python-based, easy onboarding
- [k6](#) — JavaScript-based, more mature for production-grade testing
- [NVIDIA GenAI-Perf](#) — LLM-specific benchmarking
- [GuideLLM by Red Hat](#) — new tool, works great with vLLM

I personally prefer k6 for production-grade testing. The scripting is powerful and the reporting is excellent.

Specialization 6 — System Design for ML

This is what gets you through senior interviews and what you'll actually do day-to-day as a senior. Design decisions have compounding effects; making them well is what separates senior engineers.

Skills to develop:

- Feature stores (Feast, Tecton, Hopsworks)
- Streaming ML systems (Kafka, Flink, Pulsar)
- Multi-region deployments and data residency
- A/B testing infrastructure for ML models
- Shadow deployments and canary releases
- Cost optimization at scale
- Data lineage and provenance
- ML platform design (multi-tenant, self-service)



Protection Law.

Free resources

- ["Designing Machine Learning Systems" by Chip Huyen](#) — essential. Every senior MLOps engineer I know owns this book.
- ["Machine Learning Engineering" by Andriy Burkov](#) — free PDF
- [Stanford CS329S: ML Systems Design](#) — lectures on YouTube, also taught by Chip Huyen. A distilled version of her book with real case studies.
- [Full Stack Deep Learning](#) — free comprehensive course covering the entire ML stack
- ["AI Engineering" by Chip Huyen \(2024\)](#) — her newer book, focused specifically on production LLM systems. Essential for LLMOps.

Specialization 7 — Soft Skills

Yes, really. I'll be honest — I underestimated this for years. What actually separates a senior individual contributor from a Staff+ engineer is not more tools. It's these:

- **Mentoring juniors and mid-level engineers** — you're now measured by team output, not just yours
- **Technical writing** — ADRs (Architecture Decision Records), RFCs, runbooks. Your ideas only matter if others can understand them
- **Cross-team communication** — translating between data scientists, software engineers, product, and business stakeholders
- **Cost ownership** — understanding the business impact of your technical decisions
- **Incident management** — leading post-mortems that don't blame individuals
- **Strategic thinking** — where should the platform be in 12-18 months?



- "The Staff Engineer's Path" by Tanya Reilly
- "The Pragmatic Programmer" — timeless
- "Team Topologies" by Skelton & Pais

Engineering blogs worth following: Netflix TechBlog, Uber Engineering, Spotify Engineering, DoorDash Engineering, and Pinterest Engineering.

The MENA-Specific Reality Check

Since I'm building this community for people in our region, let me be honest about what "Senior MLOps Engineer" means here.

The good news:

- There's a massive gap in senior MLOps talent in MENA. If you put in the work, opportunities are abundant.
- Salaries for senior roles in Saudi Arabia, UAE, and Qatar are competitive globally.
- Companies like Unifonic, STC, Careem, Talabat, Property Finder, Noon, and dozens of local startups are actively hiring.
- The barrier to entry is lower than in Silicon Valley, but the ceiling is also lower.

The reality:

- Most companies in the region are 12-24 months behind on MLOps maturity compared to the US/EU.
- You'll often be building things from scratch that are considered solved elsewhere.
- This is actually a good thing — you learn deeply because you have no choice.



Advanced YouTube Channels for Seniors

Beyond what I shared in the Basic roadmap, these are the channels that keep me learning:

- Andrej Karpathy
- Umar Jamil
- Yannic Kilcher
- Sebastian Raschka
- Latent Space podcast
- Dwarkesh Podcast
- AI Engineer
- Sam Witteveen (agents)

Newsletters & Blogs Every Senior Should Read

- Chip Huyen's blog
- Sebastian Raschka's "Ahead of AI"
- Lilian Weng's blog — the deepest technical writeups on the internet
- Latent Space by swyx
- The Batch by Andrew Ng
- Hugging Face blog
- MLOps Community



My Honest Take on Becoming Senior

- **Specialize, don't generalize.** Pick 2-3 of these specializations and go deep. Trying to master all seven at once is the fastest way to burn out and never actually become senior. I picked LLMOps + Model Optimization + Kubernetes as my depth areas. Yours might be different.
- **Read the papers.** Tools come and go. The papers that explain WHY things work stay relevant for years. If you can't read a paper and extract the ideas, you're stuck at a certain level.
- **Own production.** Volunteer for on-call. Lead incident response. Fix the fires nobody else wants to touch. The fastest way to senior is being the person who handles complexity when it hits.
- **Build T-shaped expertise.** Be an expert in 1-2 areas, but have working knowledge of everything else in this roadmap. Senior engineers don't say "I don't do networking" — they figure it out.
- **Mentor juniors.** Teaching forces you to understand things at a deeper level. Your seniority is measured by how you elevate others, not just your own output.
- **Stay involved in open source.** Senior engineers don't just consume tools, they shape them. Contribute code, write RFCs, propose features. An accepted PR to vLLM or Langfuse is worth more than any certification.
- **Write about what you learn.** I started writing publicly about a year ago and it accelerated my learning more than any course I ever took. Explaining forces understanding.

Other paths

Beginner → Job-ready

DevOps → MLOps



Basic MLOps Engineer Roadmap

From zero to job-ready

5 phases · 6–9 months

العربية العربية

The DevOps to MLOps

transiti

Join the Community ↗



The fastest career pivot in tech

5 phases · 3–5 months

JOIN US

Learning MLOps alone is

Thousands of engineers across MENA are already in the room — reviewing each other's code, sharing openings, and getting unblocked. It costs nothing.



WhatsApp

3,000+ members



LinkedIn

3,000+ followers



YouTube

All recordings



Discord

Chat & help



X

Follow for updates





MLOps MENA
COMMUNITY

العربية العربية



[Join the Community ↗](#)



MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)

Zomra



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Brainsmingle space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.